# Design Pattern Memento

**Intento e definizione**

L'intento del pattern Memento è catturare ed esternalizzare lo stato interno di un oggetto senza violarne l'incapsulamento. Questo permette di riportare l'oggetto a quello specifico stato in un momento successivo.

Il **problema** è che spesso è necessario registrare lo stato interno di un oggetto per implementare meccanismi di:

- **Checkpoint**: per ripristinare il sistema in caso di errore.
- **Undo** (annullamento): per permettere all'utente di tornare indietro dopo un'operazione non voluta.

**Applicabilità**

Tuttavia, gli oggetti sono progettati per incapsulare il proprio stato (rendendolo privato o protetto). Esporre questo stato tramite interfaccia pubblica per permetterne il salvataggio esterno violerebbe l'incapsulamento, compromettendo l'affidabilità e l'estensibilità del software.

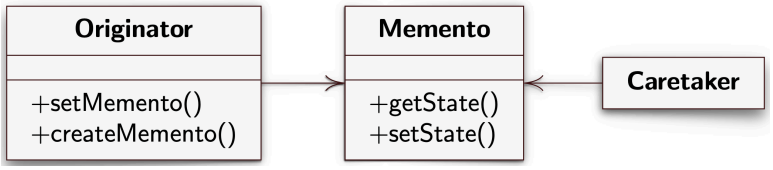
La **soluzione** che fornisce il pattern è introdurre un oggetto speciale, chiamato *Memento*, che immagazzina una copia dello stato interno dell'oggetto originale (detto *Originator*). La regola fondamentale è che solo l'Originator può accedere ai dati interni del Memento per salvare o ripristinare il proprio stato. Agli altri oggetti questo accesso è negato.

## Struttura e partecipanti

Il pattern prevede tre attori principali:

1. **Originator** (oggetto di origine):
   - è l'oggetto di cui si vuole salvare lo stato.
   - crea un Memento contenente un'istantanea del suo stato corrente, tramite createMemento().
   - usa il Memento per ripristinare il suo stato precedente, tramite setMemento().
2. **Memento**:
   - è l'oggetto "cassaforte" che immagazzina lo stato dell'Originator.
   - protegge i dati dall'accesso di qualsiasi oggetto che non sia l'Originator stesso.
3. **Caretaker** (custode):
   - è responsabile della conservazione del Memento (ad esempio, lo tiene in una lista di "storia" per l'undo).
   - non deve mai operare sui contenuti del Memento né esaminarli. Si limita a custodirlo e a restituirlo all'Originator quando richiesto.
   - inoltre, è anche responsabile della cancellazione del Memento.

## Diagramma UML delle classi




## Conseguenze

### Vantaggi

- **Preservazione dell'incapsulamento**: i dati interni non vengono esposti pubblicamente, ma rimangono confinati tra Originator e Memento.
- **Semplificazione dell'Originator**: non deve gestire internamente la cronologia delle versioni, poiché questa responsabilità è delegata al Caretaker.
- Se i salvataggi dello stato avvengono in modo ordinato e prevedibile, non serve memorizzare ogni volta tutto lo stato, ma basta salvare solo le modifiche effettive rispetto allo stato precedente. Quindi alla fine si può ricostruire lo stato completo sommando le modifiche.

### Svantaggi

- **Costo delle prestazioni**: se lo stato dell'oggetto è molto grande o se i salvataggi sono frequenti, la copia dei dati può diventare costosa in termini di memoria e tempo di elaborazione.
- **Gestione della memoria**: il Caretaker non sa quanto sia grande il Memento che sta custodendo. Questo potrebbe portare a un consumo eccessivo di memoria o spazio su disco senza che il Caretaker ne sia consapevole.

È possibile vedere un esempio al seguente link: https://github.com/isd-inf/memento/.